

UFUG 2106: Tutorial 11

Zhengquan Li

DSA, InfoHub

Apr 17, 2026

Graph

Graph:

A collection of **vertices (nodes)** connected by **edges**. Can be directed or undirected.

Types:

- **Undirected Graphs:** Edges have no direction.
- **Directed Graphs (Digraphs):** Edges have direction.
- **Weighted Graphs:** Edges have assigned weights.

Key Terms:

- **Degree:** Number of edges connected to a vertex.
- **Path:** A sequence of vertices connected by edges.
- **Cycle:** A path that starts and ends at the same vertex.

Applications:

- Social Networks
- Computer Networks
- Route Optimization

Travelling in a Graph

Key Definitions:

- **Walk:** A sequence of vertices where each pair is connected by an edge.
- **Trail:** A walk where no edge is repeated.
- **Path:** A trail where no vertex is repeated.
- **Circuit (or Cycle):** A closed trail where the start and end vertex are the same.
- **Simple Circuit:** A circuit that doesn't repeat any vertex except the starting/ending one.

Matrix Representation of Graphs

Matrix Terminology:

- Rows and Columns: Define the structure of a matrix ($m \times n$ dimensions).
- Elements: Individual values in the matrix, addressed by position (row, column).
- Diagonals: Principal diagonal (top-left to bottom-right) and secondary diagonal.
- Symmetric Matrix: A matrix is symmetric if it is equal to its transpose ($A = A^T$).

Adjacency Matrix Representation of a Graph:

- Definition: A square matrix used to represent a graph, where rows and columns correspond to graph vertices.

Entries:

- 1 if there is an edge between two vertices.
- 0 if no edge exists.

Matrix Representation of Graphs

Directed vs. Undirected Graphs:

- Directed graphs: Asymmetric adjacency matrix.
- Undirected graphs: Symmetric adjacency matrix.

Graph Connectivity:

- Connected Graph: All vertices reachable from any other vertex.
- Disconnected Graph: At least one vertex pair is not connected.

Counting Edges:

- Undirected Graph: Sum of all elements in the adjacency matrix divided by 2.
- Directed Graph: Sum of all elements in the adjacency matrix.

Checking for Loops:

- Loops exist if the diagonal elements of the adjacency matrix ($a[i][i]$) are non-zero.

Matrix Operations

Product: Row by column, **dot** product

- Result keep M, and N, while K is the reduction dim
- Dot product: multiply elementwise and accumulate.

$$c_{ij} = \sum_{t=1}^k a_{it} b_{tj}$$

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj}$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{ik} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mk} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1j} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2j} & \cdots & b_{2n} \\ \vdots & \vdots & & \vdots & & \vdots \\ \vdots & \vdots & & \vdots & & \vdots \\ \vdots & \vdots & & \vdots & & \vdots \\ b_{k1} & b_{k2} & \cdots & b_{kj} & \cdots & b_{kn} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1j} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2j} & \cdots & c_{2n} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{i1} & c_{i2} & \cdots & c_{ij} & \cdots & c_{in} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mj} & \cdots & c_{mn} \end{bmatrix}$$

Matrix multiplication & Walks

Theorem 10.2.2

If G is a graph with vertices $v_1, v_2, \dots, v_m$ and $\mathbf{A}$ is the adjacency matrix of G , then for each positive integer n and for all integers $i, j = 1, 2, \dots, m$,

the ij th entry of $\mathbf{A}^n$ = the number of walks of length n from v_i to v_j .

$$\mathbf{A} = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 & v_4 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 2 & 1 \\ 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

How many walks of length 2 are there from v_2 to v_3 ?

Isomorphisms of Graphs

- Two graphs are equivalent
- There is no more efficient general method known for checking whether two graphs are isomorphic (NP-hard)

Theorem 10.3.2

Each of the following properties is an invariant for graph isomorphism, where n , m , and k are all nonnegative integers:

1. has n vertices
2. has m edges
3. has a vertex of degree k
4. has m vertices of degree k
5. has a circuit of length k
6. has a simple circuit of length k
7. has m simple circuits of length k
8. is connected
9. has an Euler circuit
10. has a Hamiltonian circuit.

Tree

What is a Tree?

- A **tree** is a **simpler type of graph**:
 - An **undirected graph** with **no circuits** (acyclic).
 - A **connected graph**: Every node is reachable from any other node.
- Trees are hierarchical structures with **parent-child relationships** between nodes.
- Root, leaf node, internal node

Tree

Characterization of a Tree

- For a tree with n nodes:
 - It has exactly **$n-1$ edges**.
 - Adding one more edge creates a **cycle**, violating the tree property.

Depth and Height

1. Depth of a Node:

1. The number of edges from the root to the node.
2. The root has a depth of **0**.

2. Height of a Node:

1. The number of edges on the longest path from the node to a leaf.
2. The height of the **tree** is the height of the root.

Binary Tree:

- A tree where each node has at most **two children**.

Determine Whether a Graph is a Tree

Theorem 10.4.2

For any positive integer n , any tree with n vertices has $n - 1$ edges.

Theorem 10.4.4

For any positive integer n , if G is a connected graph with n vertices and $n - 1$ edges, then G is a tree.

1. Given the adjacency matrix A of a graph:

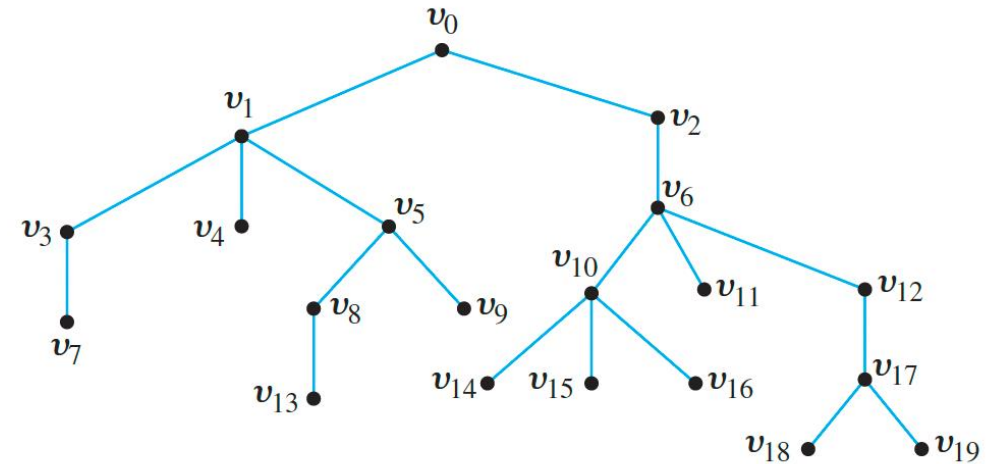
$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Is this graph a tree? Explain your answer.

The Structure of Trees

- Consider the tree shown below with root v_0 .

- What is the level of v_8 ?
- What is the level of v_0 ?
- What is the height of this rooted tree?
- What are the children of v_{10} ?
- What is the parent of v_5 ?
- What are the siblings of v_1 ?
- What are the descendants of v_{12} ?
- How many leaves (terminal vertices) are on the tree?



The Structure of Trees

- Consider the tree shown below with root v_0 .

a. What is the level of v_8 ?

3

b. What is the level of v_0 ?

0

c. What is the height of this rooted tree?

5

d. What are the children of v_{10} ?

v_{14}, v_{15}, v_{16}

e. What is the parent of v_5 ?

v_1

f. What are the siblings of v_1 ?

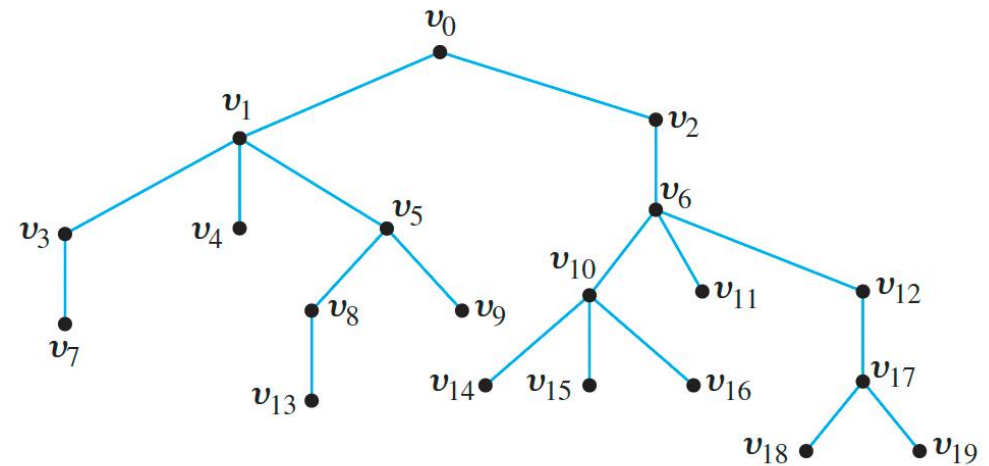
v_2

g. What are the descendants of v_{12} ?

v_{17}, v_{18}, v_{19}

h. How many leaves (terminal vertices) are on the tree?

10



Constructing Trees

- Bottom-Up
- Top-Down

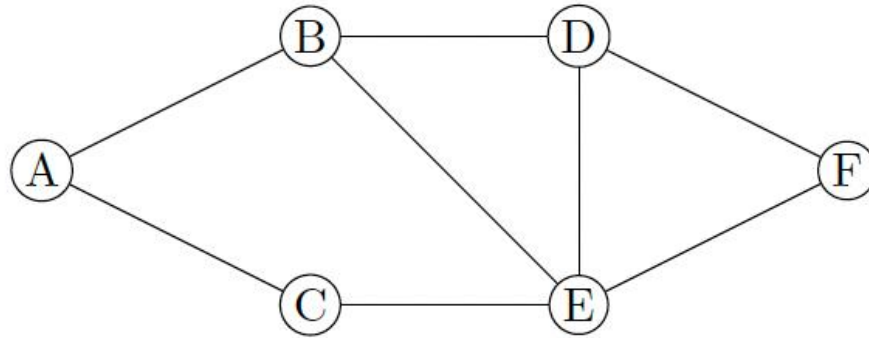
Construct the binary search tree for the sequence (16, 3, 9, 18, 11, 5).

Represent the algebraic expression $(a + b) \times (c - (d + e))$ as a binary tree.

Spanning Trees

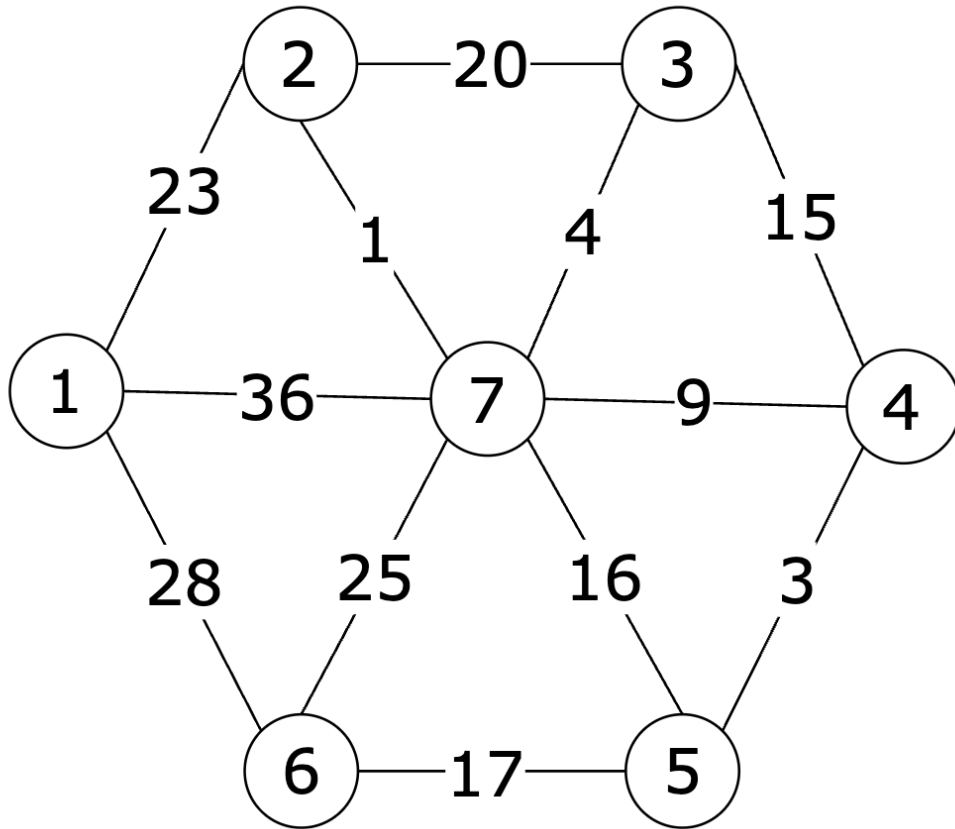
A spanning tree for a graph G is a subgraph of G that contains every vertex of G and is a tree.

5. The graph below is given. Find one of its spanning trees and mark the corresponding edges.



Tick the **edges** that you would include in the spanning tree, and mark the **root** of the tree.

Finding MST with Prim's Algorithm



Algorithm 10.6.2

Input: G [a connected, weighted graph with n vertices where n is a positive integer]

Algorithm Body:

[Build a subgraph T of G by starting with any vertex v of G and attaching edges (with their endpoints) one by one to an as-yet-unconnected vertex of G , each time choosing an edge of least weight that is adjacent to a vertex of T .]

1. Pick a vertex v of G and let T be the graph with one vertex, v , and no edges.
2. Let V be the set of all vertices of G except v .
3. **for** $i := 1$ **to** $n - 1$
 - 3a. Find an edge e of G such that (1) e connects T to one of the vertices in V , and (2) e has the least weight of all edges connecting T to a vertex in V . Let w be the endpoint of e that is in V .
 - 3b. Add e and w to the edge and vertex sets of T , and delete w from V .
- next** i

Output: T [T is a minimum spanning tree for G .]

Finding MST with Prim's Algorithm

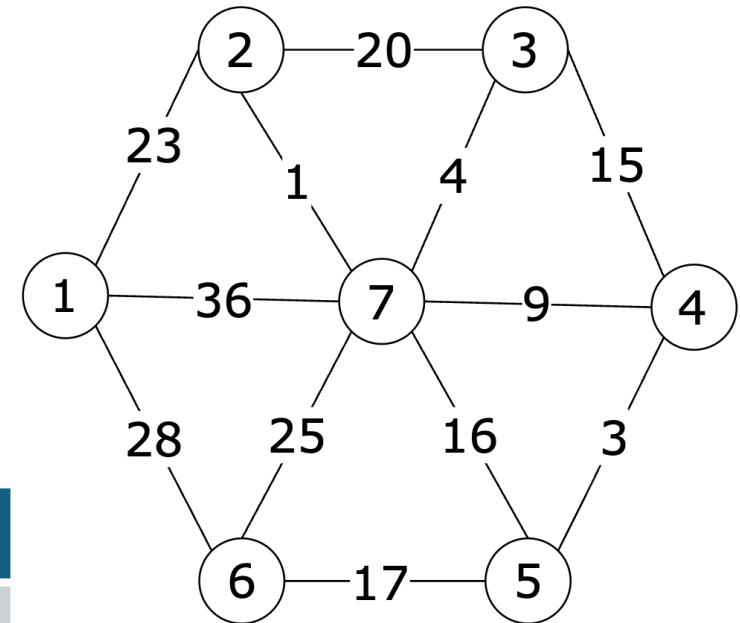
Step 0: Starting from Node 1

Update the distance for every s in S.

Track the distance between unselected nodes V, and current set S.

S	V
1	2, 3, 4, 5, 6, 7

Node	1	2	3	4	5	6	7
Dist	0	23	∞	∞	∞	28	36



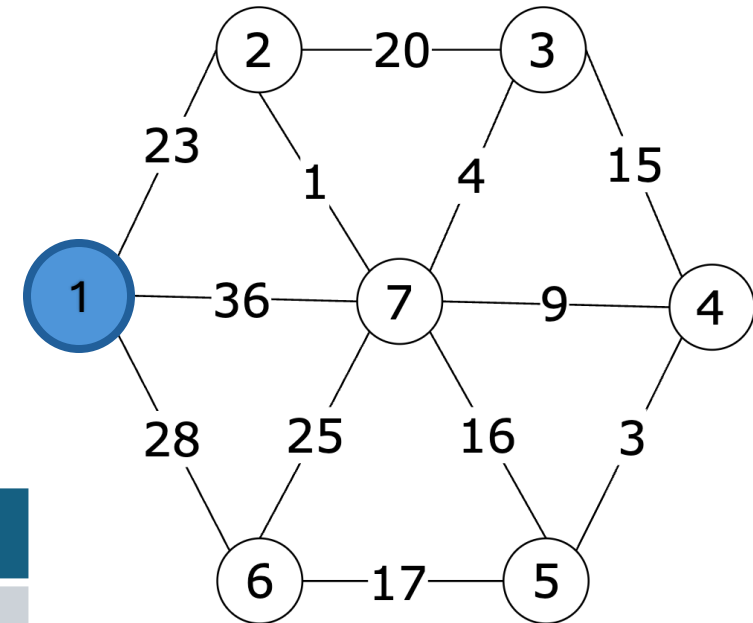
Finding MST with Prim's Algorithm

Select 2 (Nearest in V)

Update the distance for every s in S.

S	V
1, 2	2, 3, 4, 5, 6, 7

Node	1	2	3	4	5	6	7
Dist	0	23	20	∞	∞	28	1



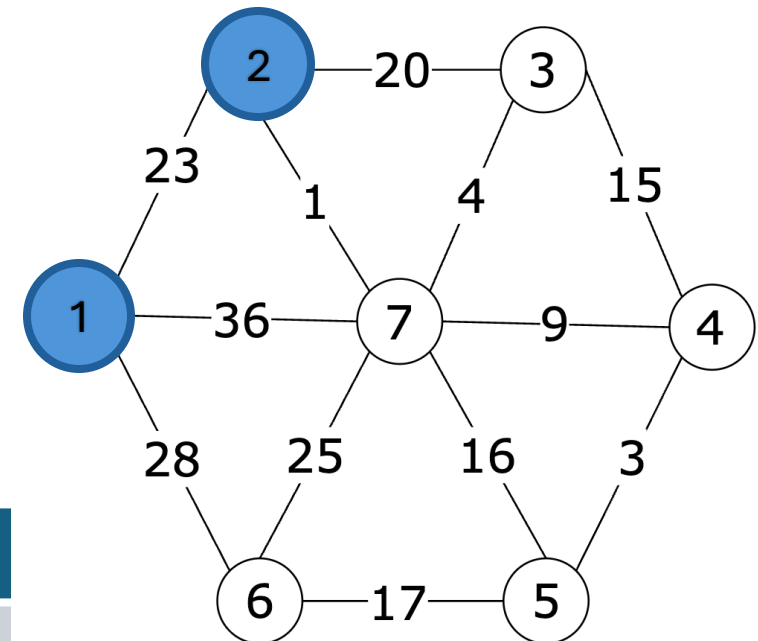
Finding MST with Prim's Algorithm

Select 7 (Nearest in V)

Update the distance for every s in S.

S	V
1, 2, 7	3, 4, 5, 6, 7

Node	1	2	3	4	5	6	7
Dist	0	23	4	9	16	25	1



Finding MST with Prim's Algorithm

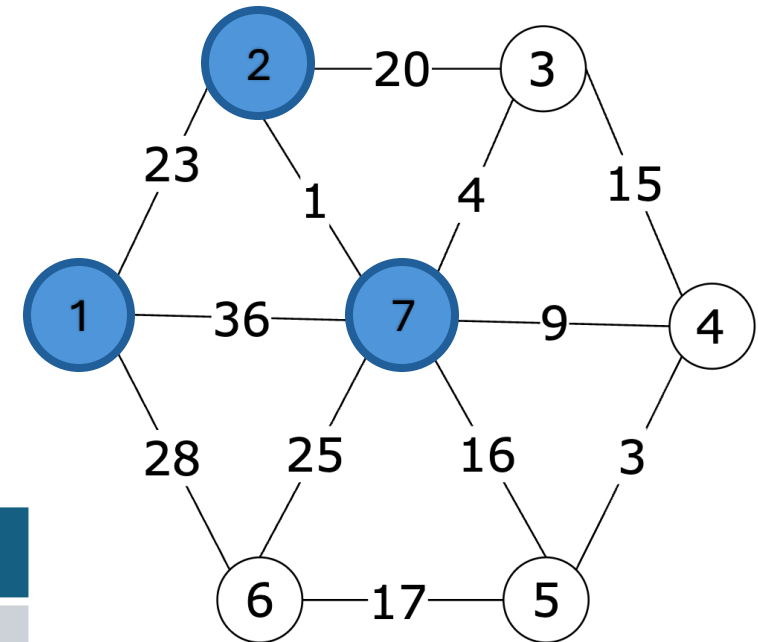
Select 3 (Nearest in V)

Update the distance for every s in S.

No Updates.

S	V
1, 2, 7, 3	3, 4, 5, 6

Node	1	2	3	4	5	6	7
Dist	0	23	4	9	16	25	1



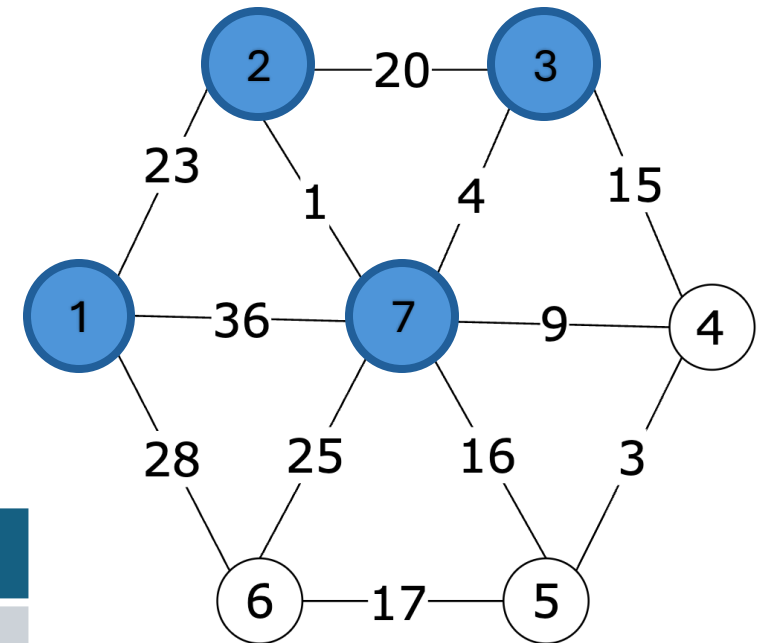
Finding MST with Prim's Algorithm

Select 4 (Nearest in V)

Update the distance for every s in S.

S	V
1, 2, 7, 3, 4	4, 5, 6

Node	1	2	3	4	5	6	7
Dist	0	23	4	9	3	25	1



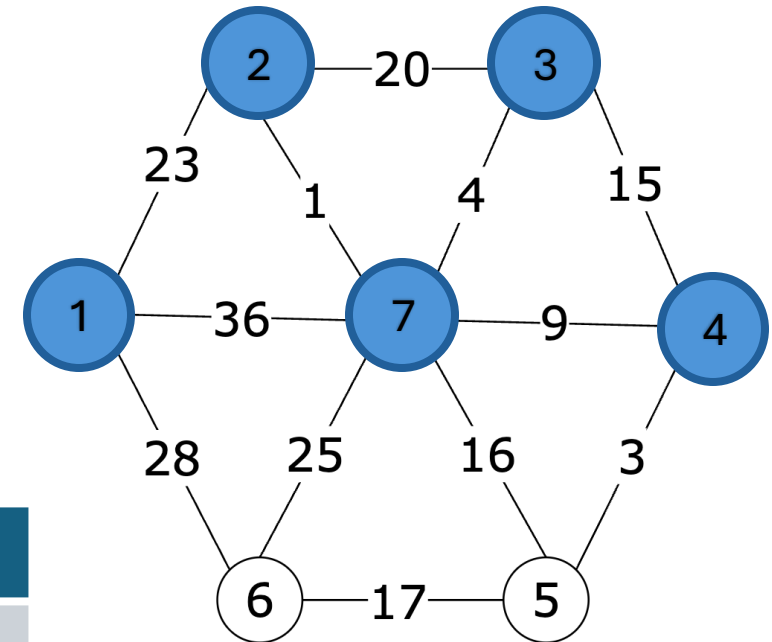
Finding MST with Prim's Algorithm

Select 5 (Nearest in V)

Update the distance

S	V
1, 2, 7, 3, 4, 5	5, 6

Node	1	2	3	4	5	6	7
Dist	0	23	4	9	3	17	1



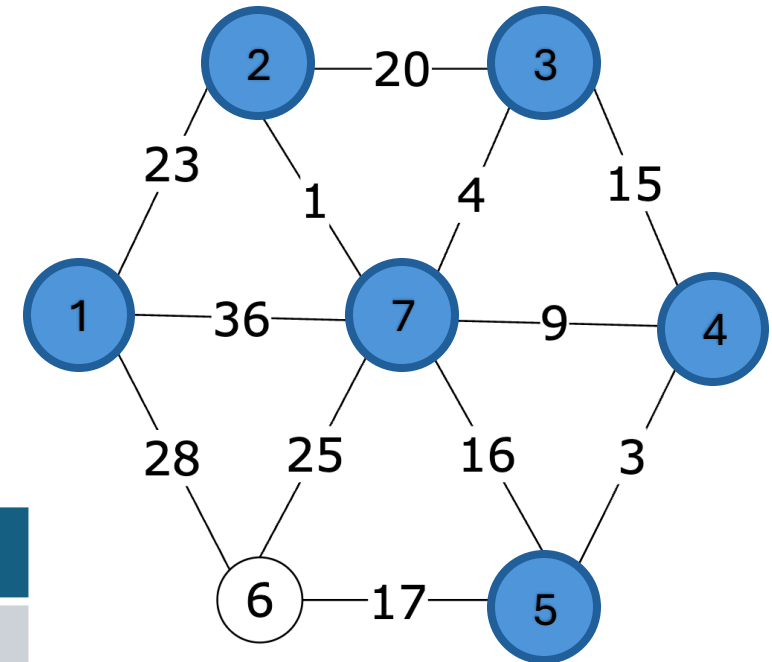
Finding MST with Prim's Algorithm

Select 6

Terminates.

S	V
1, 2, 7, 3, 4, 5, 6	6

Node	1	2	3	4	5	6	7
Dist	0	23	4	9	3	17	1



Kruskal's Algorithm

Algorithm 10.6.1 Kruskal

Input: G [a connected, weighted graph with n vertices, where n is a positive integer]

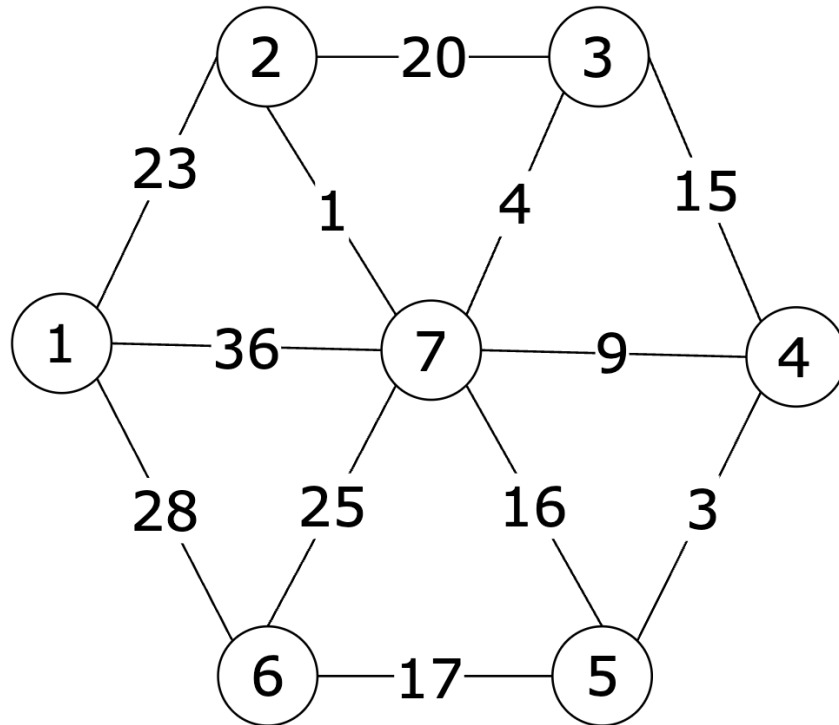
Algorithm Body:

[Build a subgraph T of G to consist of all the vertices of G with edges added in order of increasing weight. At each stage, let m be the number of edges of T .]

1. Initialize T to have all the vertices of G and no edges.
 2. Let E be the set of all the edges of G , and let $m := 0$.
 3. **while** ($m < n - 1$)
 - 3a. Find an edge e in E of least weight.
 - 3b. Delete e from E .
 - 3c. **if** addition of e to the edge set of T does not produce a circuit
 then add e to the edge set of T and set $m := m + 1$
- end while**

Output: T [T is a minimum spanning tree for G .]

Finding MST with Kruskal's Algorithm



2-7: 1
4-5: 3
3-7: 4
4-7: 9
3-4: 15
5-7: 16
5-6: 17
2-3: 20
1-2: 23
7-6: 25
1-6: 28
1-7: 36

Key Takeaways

Prim's Algorithm:

- Builds the Minimum Spanning Tree (MST) incrementally starting from an arbitrary node.
- Always grows the MST by adding the smallest edge connecting the tree to a new vertex.
- Uses a priority queue or min-heap for efficient edge selection.
- Best suited for dense graphs.

Kruskal's Algorithm:

- Sorts all edges by weight and adds them to the MST if they don't form a cycle.
- Utilizes Union-Find (Disjoint Set) for cycle detection.
- Works well for sparse graphs due to edge-based processing.
- Requires sorting, making it efficient for graphs with fewer edges.

Comparison:

- **Prim's** is vertex-focused; **Kruskal's** is edge-focused.
- Both guarantee the same MST for a connected, weighted graph.

Shortest Path Finding: Dijkstra's Algorithm

- A greedy algorithm for finding the shortest path in a weighted graph.
- Works for graphs with non-negative weights.

Algorithm 10.6.3 Dijkstra

4. while ($z \notin V(T)$)

4a. $F := (F - \{v\}) \cup \{\text{vertices that are adjacent to } v \text{ and are not in } V(T)\}$
[The set F is called the fringe. Each time a vertex is added to T , it is removed from the fringe and the vertices adjacent to it are added to the fringe if they are not already in the fringe or the tree T .]

4b. For each vertex u that is adjacent to v and is not in $V(T)$,
if $L(v) + w(v, u) < L(u)$ **then**

$L(u) := L(v) + w(v, u)$

$D(u) := v$

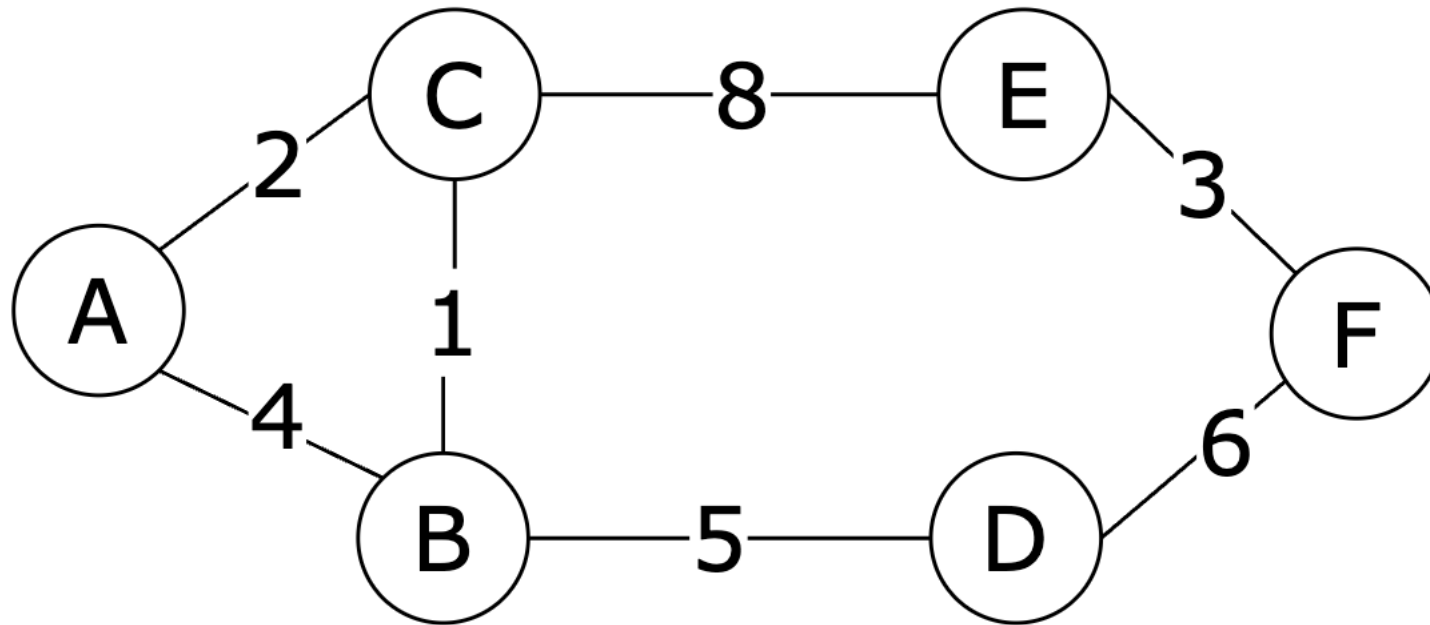
[Note that adding v to T does not affect the labels of any vertices in the fringe F except those adjacent to v . Also, when $L(u)$ is changed to a smaller value, the notation $D(u)$ is introduced to keep track of which vertex in T gave rise to the smaller value.]

4c. Find a vertex x in F with the smallest label
Add vertex x to $V(T)$, and add edge $\{D(x), x\}$ to $E(T)$
 $v := x$ *[This statement sets up the notation for the next iteration of the loop.]*

end while

Output: $L(z)$ *[$L(z)$, a nonnegative integer, is the length of the shortest path from a to z .]*

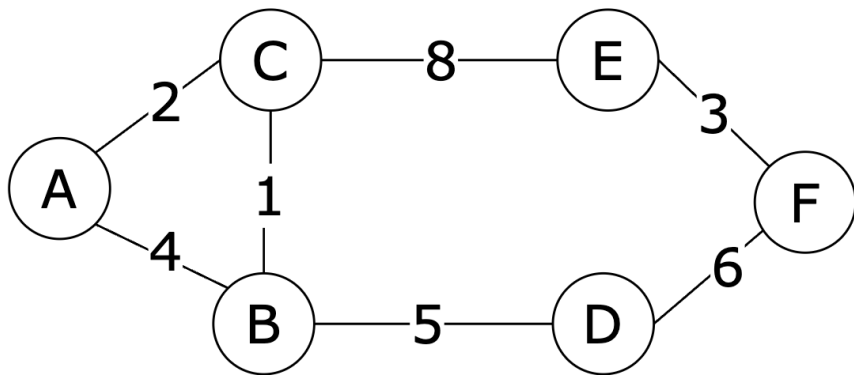
Dijkstra's Algorithm: Example



Dijkstra's Algorithm: Example

Step 1:

- Mark all nodes as unvisited
- Set the distance to the starting node (A) as **0** and the distance to all other nodes as **infinity**.

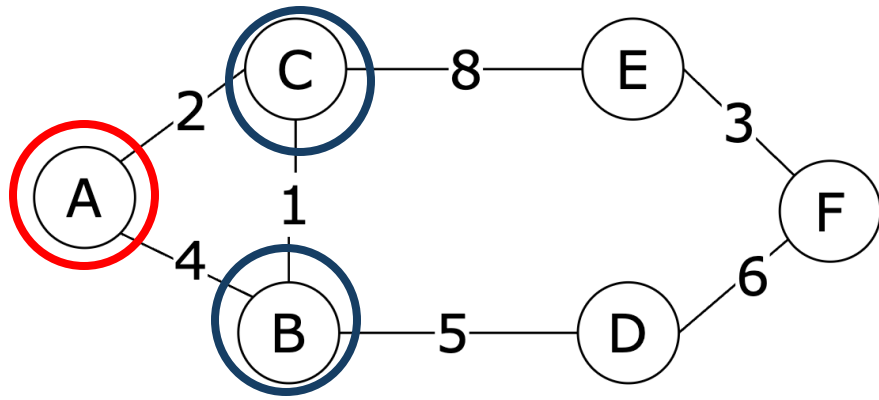


Node	Distance	Prev
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-

Dijkstra's Algorithm: Example

Step 2: From A, update distances to its neighbors:

- Distance to B = $0 + 4 = 4$
- Distance to C = $0 + 2 = 2$

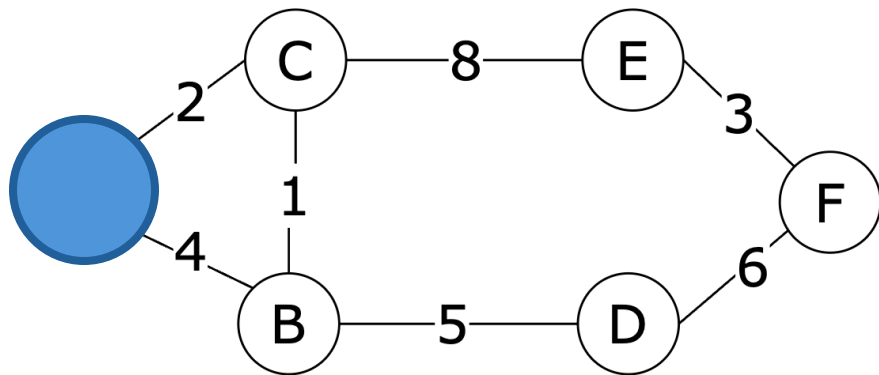


Node	Distance	Prev
A	0	-
B	4	A
C	2	A
D	∞	-
E	∞	-
F	∞	-

Dijkstra's Algorithm: Example

Step 2: From A, update distances to its neighbors:

- Distance to B = $0 + 4 = 4$
- Distance to C = $0 + 2 = 2$
- Mark A as visited

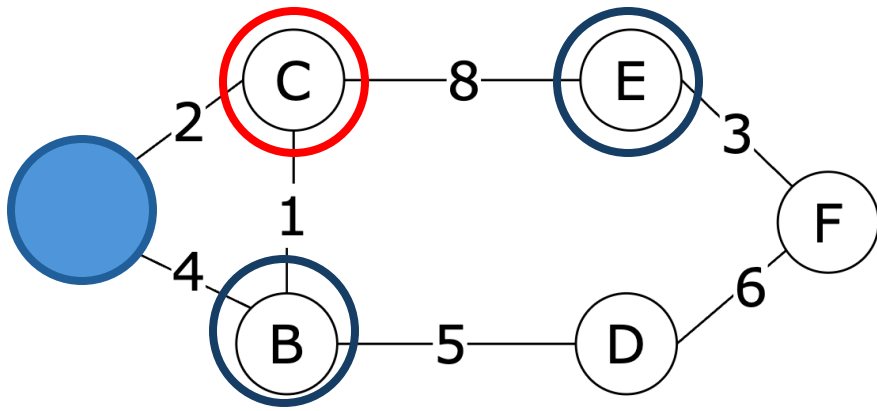


Node	Distance	Prev
A	0	-
B	4	A
C	2	A
D	∞	-
E	∞	-
F	∞	-

Dijkstra's Algorithm: Example

Step 2: Pick the nearest **unvisited** destination in the table, which is C, update the corresponding distances

- Distance to B = $2 + 1 = 3$
- Distance to E = $2 + 8 = 10$
- Mark C as visited

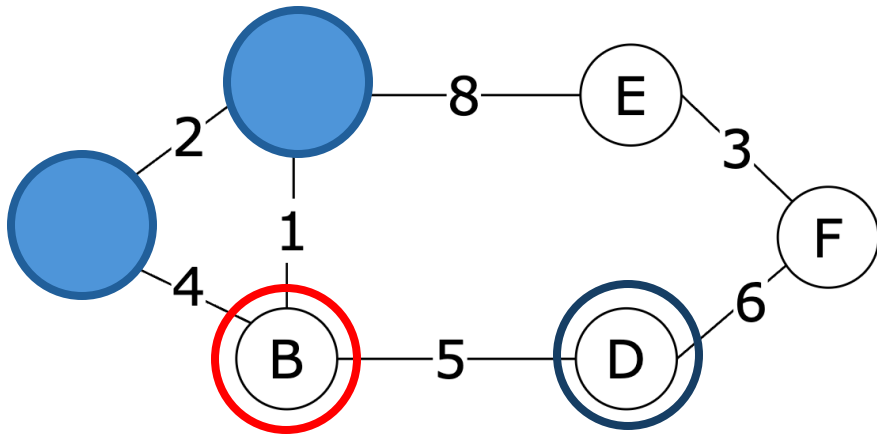


Node	Distance	Prev
A	0	-
B	3	C
C	2	A
D	∞	-
E	10	C
F	∞	-

Dijkstra's Algorithm: Example

Step 3: Pick the nearest **unvisited** destination in the table, which is B, update the corresponding distances

- Distance to D = $3 + 5 = 8$
- Mark B as visited

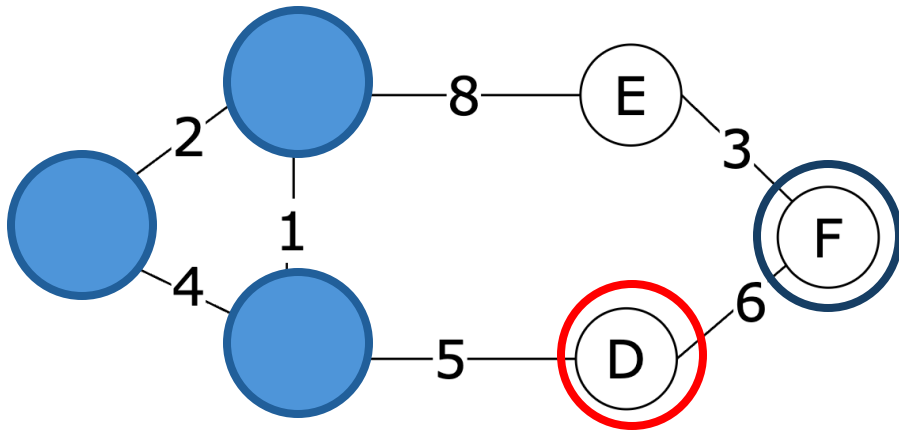


Node	Distance	Prev
A	0	-
B	3	C
C	2	A
D	8	B
E	10	C
F	∞	-

Dijkstra's Algorithm: Example

Step 4: Pick the nearest **unvisited** destination in the table, which is D, update the corresponding distances

- Distance to F = $8 + 6 = 14$
- Mark D as visited

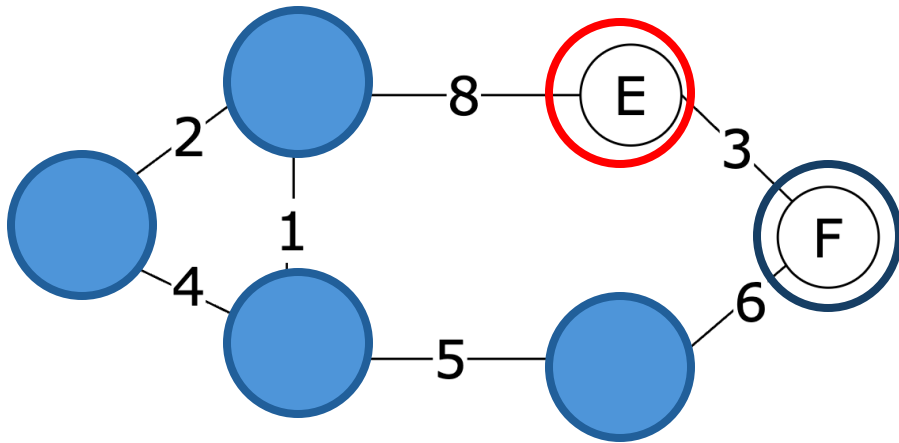


Node	Distance	Prev
A	0	-
B	3	C
C	2	A
D	8	B
E	10	C
F	14	D

Dijkstra's Algorithm: Example

Step 5: Pick the nearest **unvisited** destination in the table, which is E, update the corresponding distances

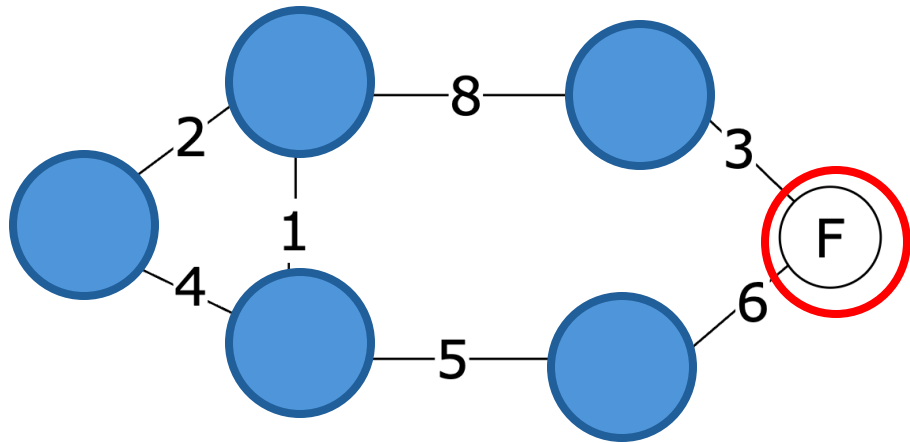
- Distance to F = $10 + 3 = 13$
- Mark E as visited



Node	Distance	Prev
A	0	-
B	3	C
C	2	A
D	8	B
E	10	C
F	13	E

Dijkstra's Algorithm: Example

Last Step: Visit F, no other unvisited nodes.
Terminated.



Node	Distance	Prev
A	0	-
B	3	C
C	2	A
D	8	B
E	10	C
F	13	E

Shortest Path Finding: Dijkstra's Algorithm

- Key takeaways:
 - No backwards steps: Once a node is visited, it will not be considered again. The algorithm does not revisit nodes.
 - Greedy: Always chooses the unvisited node with the shortest distance to the origin to process next.
 - Works only with graphs that have non-negative edge weights (negative weights require a different algorithm).
 - Optimal structure: Guarantees the shortest path to each node as it progresses, building the solution incrementally.

Prim, Dijkstra, Similar?

Objective:

- Prim's: Builds a **tree** that spans all nodes with the minimum total weight.
- Dijkstra's: Finds the **shortest path** from a source to a destination.

Selection Criteria:

- Prim's: Chooses the **shortest edge** connecting visited and unvisited nodes.
 - If(!visited[v] && G[u][v] < distance[v])
- Dijkstra's: Chooses the **node** with the smallest distance from the source.
 - If(!visited[v] && G[u][v] + distance[u] < distance[v])

Data Representation:

- Prim's: dis[i] represents the shortest edge connecting node i to the visited set.
- Dijkstra's: dis[i] represents the shortest distance from the source to node i.

Shortest Path Finding: A* Algorithm

- Heuristic algorithm
 - **Actual cost** from the start node ($g(x)$).
 - **Heuristic estimate** to the goal node ($h(x)$).
- A heuristic is a **rule of thumb** or **educated guess** that helps guide the search.
 - In A*, the heuristic estimates the shortest distance from the current node to the goal.
 - Example: Straight-line distance is often used as a heuristic in maps.

A* Algorithm: Main Idea

- Input:
 - A graph with:
 - **Nodes** and **weighted edges**.
 - A **start node** and a **goal node**.
 - A **heuristic function ($h(x)$)**: Estimates the cost from a node to the goal.
- **$f(x) = g(x) + h(x)$**
 - $g(x)$: Actual cost from the start node to the current node.
 - $h(x)$: Heuristic estimate to the goal.
 - **$f(x)$: Total estimated cost.**

A* Algorithm: Open List / Closed List

Open list: The open list contains nodes that are **discovered but not yet fully explored**.

- These are the nodes that are candidates for expansion in the next steps of the algorithm.

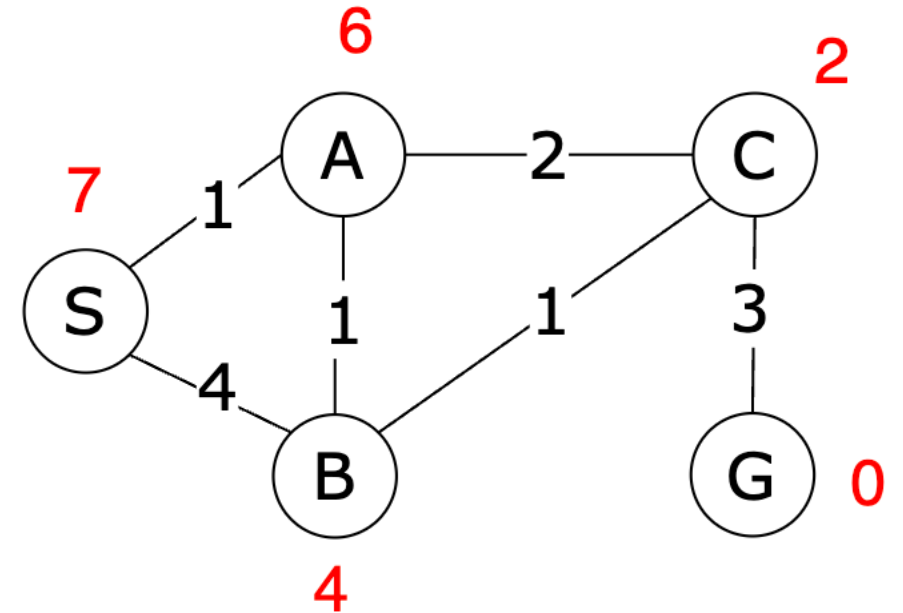
Closed list: The closed list contains nodes that have already been **visited and fully explored**. Once a node is placed in the closed list, it will not be revisited.

- The closed list ensures that the algorithm does not revisit nodes unnecessarily, preventing cycles and redundant work.

A* Algorithm: Example

- The definition of $h(x)$ is omitted for simplicity
- Start from S
- $g(x)$: Actual cost from S to current node

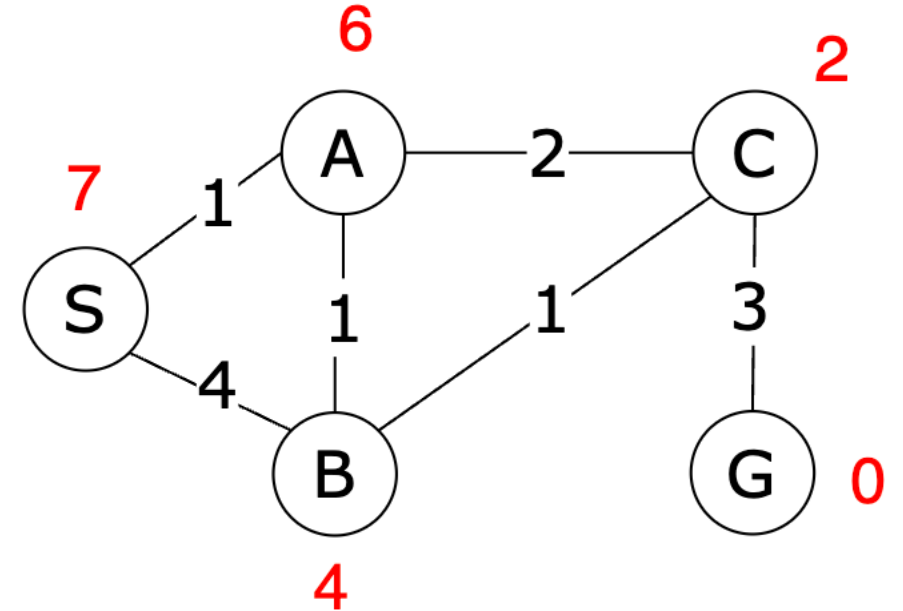
Open List	Closed List
S, $g=0$, $h=7$, $f=7$	



A* Algorithm: Example

- Step 1: Expand S
 - Candidates: A and B
- $f(A) = h(A) + g(A) = 1 + 6 = 7$
- $f(b) = h(B) + g(B) = 4 + 4 = 8$
- Move S to closed list

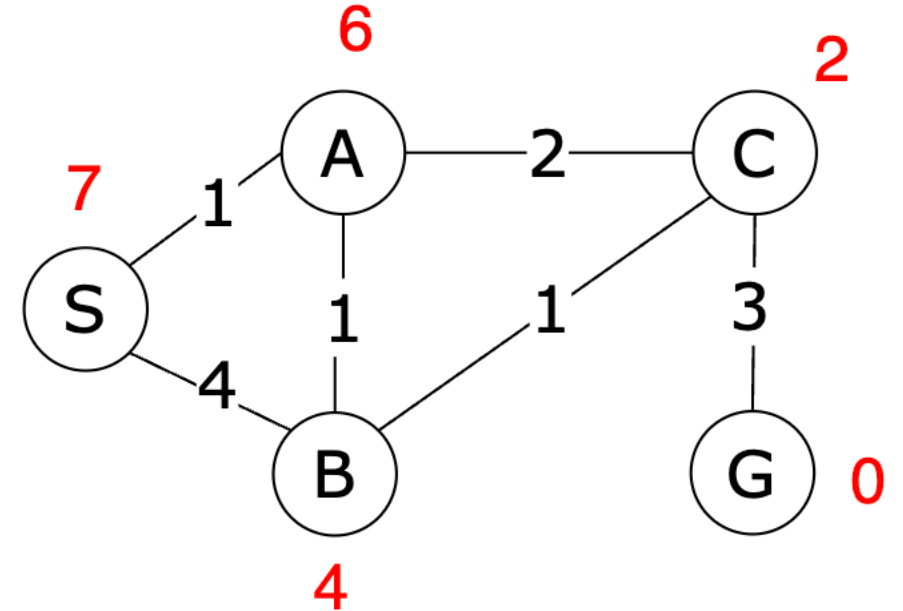
Open List	Closed List
S, g=0, h=7, f=7 A, g=1, h=6, f=7 B, g=4, h=4, f=8	S



A* Algorithm: Example

- Step 2: Expand A
 - Candidates: C
 - $f(C) = h(C) + g(C)$
 - $g(C) = g(A) + \text{cost}(AC) = 3$
 - ➔ $f(C) = 5$

Open List	Closed List
A, g=1, h=6, f=7 C, g=3, h=2, f=5 B, g=4, h=4, f=8	S A



A* Algorithm: Example

- Step 3: Expand C

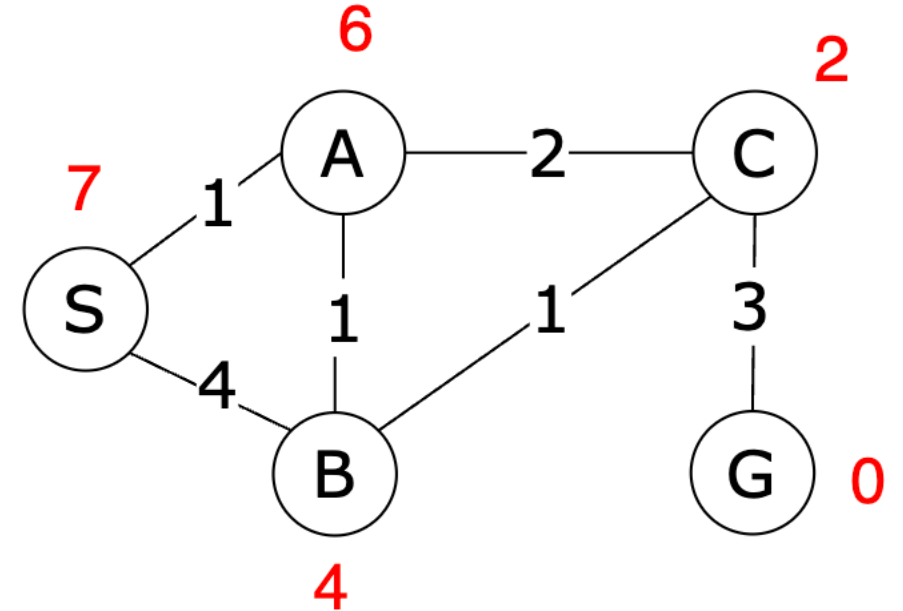
- Candidates: G

$$f(G) = h(G) + g(G)$$

$$g(G) = g(C) + \text{cost}(CG) = 3 + 3 = 6$$

$$\rightarrow f(G) = 6$$

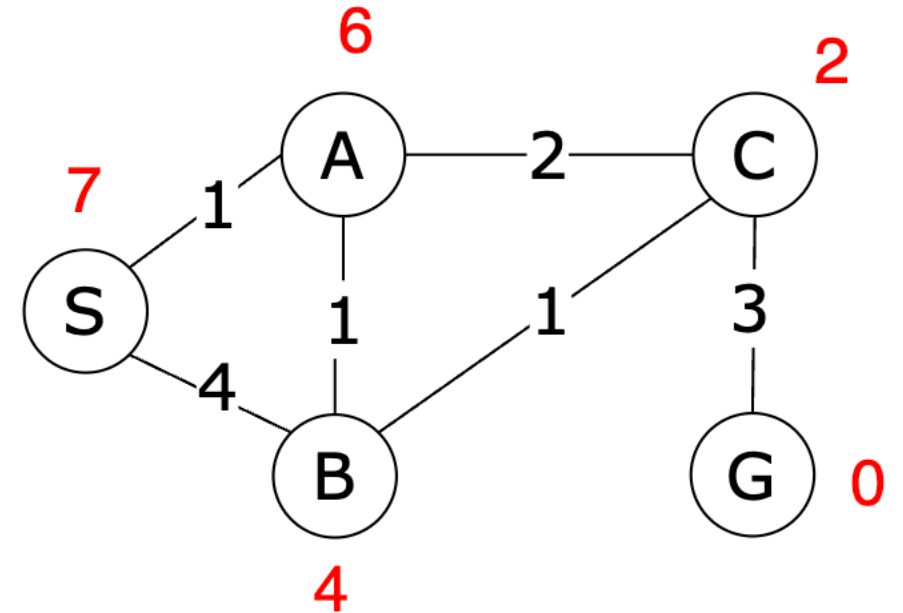
Open List	Closed List
C, g=3, h=2, f=5	S
G, g=6, h=0, f=6	A
B, g=4, h=4, f=8	C



A* Algorithm: Example

- Step 4: Expand G
Done!

Open List	Closed List
G, g=6, h=0, f=6 B, g=4, h=4, f=8	S A C G



Problem: $h(x)$?

Manhattan Distance

- Formula: $|x_1 - x_2| + |y_1 - y_2|$
- Best for grid-based graphs with horizontal and vertical movements.

Euclidean Distance

- Formula: $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- Ideal for systems allowing diagonal or straight-line movements.

Chebyshev Distance

- Formula: $\max(|x_1 - x_2|, |y_1 - y_2|)$
- Useful when diagonal and straight movements cost the same.

Zero Heuristic

- $h(n) = 0$ for all nodes.
- Reduces A* to Dijkstra's algorithm, ensuring correctness but with slower performance.

Design Principle of $h(x)$

In the context of A^* , the **triangle inequality** ensures that the heuristic function $h(n)$ satisfies the following condition for consistency:

For any two nodes n and m , the heuristic estimate $h(n)$ must be less than or equal to the actual cost of moving from n to m plus the heuristic estimate of m :

$$h(n) \leq \text{Cost}(n, m) + h(m)$$

What This Means for A^* :

- 1.If $h(n)$ satisfies the triangle inequality, A^* guarantees that once it finds a path to a node, that path is the shortest possible path.
- 2.This ensures that A^* always expands nodes in the correct order, leading to correctness and efficiency.

Example: Euclidean distance

- $N(0, 0)$ $M(3,4)$
- Actual cost ≥ 5 , assume $\text{cost}(M,N)=6$
- $h(N) = 0$
- $h(M) = 5$

$$h(n) \leq \text{Cost}(n, m) + h(m)$$

Comparison

- Comparison with Dijkstra's algorithm

	Dijkstra's Algorithm	A* Algorithm
Use Heuristic?	No. Only actual cost ($g(x)$)	Use $h(x)$ to guide the search
Exploration	All possible paths	Focus on paths likely leading to the goal
Efficiency	Slower, considers unnecessary nodes.	Faster, avoids unnecessary exploration.

Thank you!